

```
# Downloading a compressed version of the CSV data file from Open Food Facts.
!wget -q https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz -O food.csv.gz
```

```
# Importing the required libraries.
import pandas as pd
import matplotlib.pyplot as plt

# Storing only the relevant columns needed for the analysis based on the column names provided in the metadata on
columns_needed = [
    'code',
    'product_name',
    'categories_tags',
    'categories_en',
    'ingredients_text',
    'sugars_100g',
    'proteins_100g',
    'fat_100g',
    'fiber_100g',
    'carbohydrates_100g',
    'energy-kcal_100g',
    'nutriscore_grade',
]

df = pd.read_csv(
    "food.csv.gz",
    sep="\t", # The character that separates fields is <tab> (tabulation) according to the metadata provided by
    compression="gzip",
    nrows=500_000,
    usecols=columns_needed, # Only pulls the needed libraries
    low_memory=False,
    on_bad_lines="skip"
)

display(df.head())
print("The rows and columns in the dataset is:", {df.shape})
```

	code	product_name	categories_tags	categories_en	ingredients_text	nutriscore_grade	energy-kcal_100g	fat_100g
0	54	Limonade artisanale a la rose	NaN	NaN	NaN	unknown	NaN	NaN
1	63	M&M white	NaN	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...	unknown	NaN	NaN
2	114	Chocolate n3	NaN	NaN	NaN	unknown	NaN	NaN
3	431	Pâte de fruits	NaN	NaN	NaN	unknown	NaN	NaN
4	105	Paleta gran reserva - Sierra nevada-	en:beverages-and-beverages-preparations,en:bev...	Beverages and beverages preparations,Beverages	Thiamin, Biotin, Chromium, Garcinia cambogia f...	unknown	NaN	NaN

The rows and columns in the dataset is: {(500000, 12)}

Story 1: Data Ingestion & "The Clean Up"

Investigating The Missing Values In the Columns

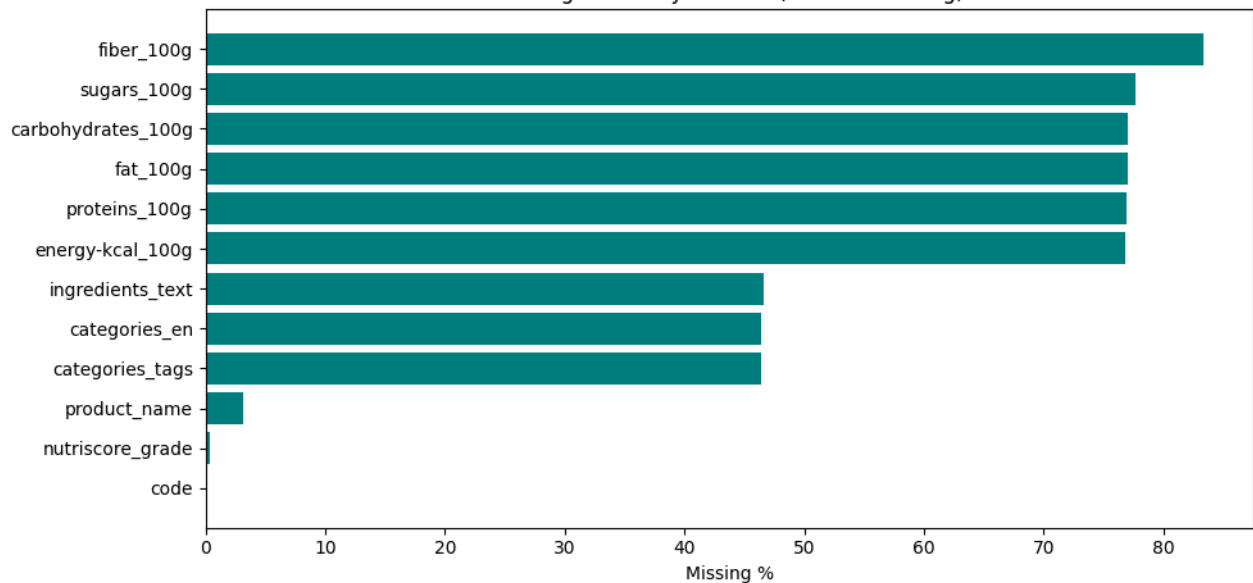
```
missing_values_summary = df.isnull().sum().reset_index()
missing_values_summary.columns = ['column', 'missing_count']
missing_values_summary['missing_values_percentage'] = (missing_values_summary['missing_count'] / len(df) * 100).round(2)
missing_values_summary = missing_values_summary.sort_values('missing_values_percentage', ascending=False)

display(missing_values_summary)

# Bar chart to display the missing values
plt.figure(figsize=(10, 5))
plt.barh(missing_values_summary['column'], missing_values_summary['missing_values_percentage'], color='teal')
plt.xlabel('Missing %')
plt.title('Missing Values by Column (Before Cleaning)')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

	column	missing_count	missing_values_percentage	
10	fiber_100g	416780	83.36	
9	sugars_100g	388365	77.67	
8	carbohydrates_100g	384967	76.99	
7	fat_100g	384924	76.98	
11	proteins_100g	384775	76.96	
6	energy-kcal_100g	383898	76.78	
4	ingredients_text	233042	46.61	
3	categories_en	231922	46.38	
2	categories_tags	231922	46.38	
1	product_name	15789	3.16	
5	nutriscore_grade	1651	0.33	
0	code	0	0.00	

Missing Values by Column (Before Cleaning)



Next steps: [Generate code with missing_values_summary](#) [New interactive sheet](#)

```
# Checking for missing values in the core columns(columns relevant for the analysis)
core_columns = ['product_name', 'sugars_100g', 'proteins_100g', 'categories_tags']
df_clean = df.dropna(subset=core_columns).copy()

print(f"Before: {len(df):,} rows, After dropping missing core fields: {len(df_clean):,} rows")
display(df_clean.head())
```

Before: 500,000 rows, After dropping missing core fields: 52,857 rows

	code	product_name	categories_tags	categories_en	ingredients_text	nutriscore_grade
689	1000	Pinto Bean	en:asian-style-ready-meal	Asian-style-ready-meal	NaN	unknown
840	1001	pasta	en:beverages-and-beverages-preparations,en:bev...	Beverages and beverages preparations,Beverages	NaN	c
865	100810429	Eirn original curry Sauce	en:plant-based-foods-and-beverages,en:plant-ba...	Plant-based foods and beverages,Plant-based fo...	Wheat Flour Sugar Vegetable Fat D Curry Powder...	not-applicable
875	101019680	Donut Milka	en:snacks,en:sweet-snacks,en:biscuits-and-cake...	Snacks,Sweet snacks,Biscuits and cakes,Cakes,D...	NaN	e
890	101209159	Véritable pâte à tartiner noisettes chocolat noir	en:breakfasts,en:spreads,en:sweet-spreads,fr:p...	Breakfasts,Spreads,Sweet spreads,fr:Pâtes à ta...	NaN	e

```
nutrient_cols = ['sugars_100g', 'proteins_100g', 'fat_100g', 'fiber_100g', 'carbohydrates_100g']
```

```
# Summary statistics for each nutrient column
print("Summary statistics for nutrient columns:")
display(df_clean[nutrient_cols].describe().T)

# Preview rows that still have at least one missing value among the nutrient columns
print("\n Sample rows with missing nutrient values:")
display(df_clean.loc[df_clean[nutrient_cols].isnull().any(axis=1)][nutrient_cols].head())

# Count of missing values per nutrient column
print("\n Missing value counts per nutrient column:")
display(df_clean[nutrient_cols].isnull().sum())
```

Summary statistics for nutrient columns:

	count	mean	std	min	25%	50%	75%	max	
sugars_100g	52857.0	13.399956	175.714334	0.0	1.176471	4.800000	14.060000	40000.0	
proteins_100g	52857.0	8.608032	39.297114	0.0	2.200000	6.600000	10.398509	6400.0	
fat_100g	52788.0	11.522648	30.232196	0.0	0.519758	3.800000	15.300000	2520.0	
fiber_100g	43356.0	3.053406	7.481829	0.0	0.000000	1.714286	3.797468	800.0	
carbohydrates_100g	52552.0	35.316082	179.477635	0.0	7.893525	25.000000	57.000000	40000.0	

Sample rows with missing nutrient values:

	sugars_100g	proteins_100g	fat_100g	fiber_100g	carbohydrates_100g
875	18.0	6.000000	28.000000	NaN	46.5
890	32.0	8.000000	48.000000	NaN	36.0
946	0.0	0.000000	36.700001	NaN	0.0
947	4.1	22.299999	35.299999	NaN	5.1
954	0.2	40.500000	22.700000	NaN	2.4

Missing value counts per nutrient column:

	0
sugars_100g	0
proteins_100g	0
fat_100g	69
fiber_100g	9501
carbohydrates_100g	305

dtype: int64

```
# Outliers for each nutrient column
for col in nutrient_cols:
    outliers = df_clean[(df_clean[col] > 100) | (df_clean[col] < 0)][['product_name', col]] \
        .sort_values(col, ascending=False)
    print(f"\n{col}: {len(outliers)} outlier rows")
    display(outliers.head(5))
```

sugars_100g: 70 outlier rows

	product_name	sugars_100g	
154062	solgar	40000.0	
273906	Lemon-Lime Soda	2400.0	
187909	Starburst Original	1600.0	
390198	Cottage Cheese Small Curd	1600.0	
46085	Snack Medleys	900.0	

proteins_100g: 48 outlier rows

	product_name	proteins_100g
390198	Cottage Cheese Small Curd	6400.0
263127	Chicken korma and pilau rice	3190.0
64317	Chicken & Spinach Garlic Alfredo Bowl	3100.0
292026	PLANT-BASED PROTEIN SHAKE	2500.0
340518	Pea Soup with Smoked Ham	900.0

fat_100g: 111 outlier rows

	product_name	fat_100g
263127	Chicken korma and pilau rice	2520.0
2798	Tester	2321.0
86538	Ghee Clarified Butter	2000.0
286208	CLARIFIED BUTTER (ghee)	2000.0
390198	Cottage Cheese Small Curd	1600.0

fiber_100g: 7 outlier rows

	product_name	fiber_100g
340518	Pea Soup with Smoked Ham	800.000000
418625	Mission zero carb tortillas	700.000000
215677	Baking Cocoa	400.000000
307426	ASHWAGANDHA+ GUMMIES	300.000000
110435	Premium Popcorn Movie Theater Butter	133.333333

carbohydrates_100g: 206 outlier rows

	product_name	carbohydrates_100g
154062	solgar	40000.0
340518	Pea Soup with Smoked Ham	2700.0
187909	Starburst Original	2400.0
273906	Lemon-Lime Soda	2400.0
2798	Tester	2312.0

```
row_len_before = len(df_clean)
```

```
for col in nutrient_cols:  
    df_clean = df_clean[df_clean[col].isnull() | ((df_clean[col] >= 0) & (df_clean[col] <= 100))]
```

```
print(f"Row length before removing outliers: {row_len_before:,} After outlier removal: {len(df_clean):,} (removed  
display(df_clean.head())
```

Row length before removing outliers: 52,857 After outlier removal: 52,565 (removed 292)

	code	product_name	categories_tags	categories_en	ingredients_text	nutriscore_grade
689	1000	Pinto Bean	en:asian-style-ready-meal	Asian-style-ready-meal	NaN	unknown
840	1001	pasta	en:beverages-and-beverages-preparations,en:bev...	Beverages and beverages preparations,Beverages	NaN	c
865	100810429	Eirn original curry Sauce	en:plant-based-foods-and-beverages,en:plant-ba...	Plant-based foods and beverages,Plant-based fo...	Wheat Flour Sugar Vegetable Fat D Curry Powder...	not-applicable
875	101019680	Donut Milka	en:snacks,en:sweet-snacks,en:biscuits-and-cake...	Snacks,Sweet snacks,Biscuits and cakes,Cakes,D...	NaN	e
890	101209159	Véritable pâte à tartiner noisettes chocolat noir	en:breakfasts,en:spreads,en:sweet-spreads,fr:p...	Breakfasts,Spreads,Sweet spreads,fr:Pâtes à ta...	NaN	e

```
# Track row count before the filtering step is performed, to report how many rows were removed before = len(df_clean)
```

```
# Keep rows where carbohydrates is missing or,
# where sugar is not greater than carbohydrates ,
# since sugar is a subset of carbohydrates and should never exceed it biologically
df_clean = df_clean[
    df_clean['carbohydrates_100g'].isnull() |
    (df_clean['sugars_100g'] <= df_clean['carbohydrates_100g'] + 1)
]
```

```
# Reports how many rows were dropped for having sugar values impossibly higher than carbs
print(f"Removed {before - len(df_clean):,} rows where sugar > carbohydrates")
```

Removed 120 rows where sugar > carbohydrates

Story 2: Category Wrangler

```
# See the most common category tags to be used for the keyword rules
from collections import Counter

all_tags = df_clean['categories_tags'].str.split(',').explode()
tag_counts = Counter(all_tags)

top_tags = pd.DataFrame(tag_counts.most_common(40), columns=['tag', 'count'])
display(top_tags)
```

	tag	count	
0	en:plant-based-foods-and-beverages	21686	
1	en:plant-based-foods	20114	
2	en:cereals-and-potatoes	12482	
3	en:snacks	9036	
4	en:breads	7143	
5	en:dairies	6766	
6	en:sweet-snacks	6018	
7	en:fermented-foods	5947	
8	en:fermented-milk-products	5893	
9	en:desserts	5361	
10	en:cereals-and-their-products	5024	
11	en:dairy-desserts	4103	
12	en:fermented-dairy-desserts	4022	
13	en:yogurts	3967	
14	en:beverages	3672	
15	en:condiments	3606	
16	en:breakfasts	3325	
17	en:fruits-and-vegetables-based-foods	3038	
18	en:sauces	2975	
19	en:meals	2747	
20	en:biscuits-and-cakes	2603	
21	en:breakfast-cereals	2468	
22	en:meats-and-their-products	2355	
23	en:confectioneries	2213	
24	en:salty-snacks	2173	
25	en:frozen-foods	2148	
26	en:appetizers	1983	
27	en:spreads	1899	
28	en:groceries	1849	
29	en:cheeses	1787	
30	en:plant-based-beverages	1702	
31	en:biscuits	1630	
32	en:meats	1629	
33	en:vegetables-based-foods	1568	
34	en:undefined	1537	
35	en:beverages-and-beverages-preparations	1535	
36	en:canned-foods	1447	
37	en:legumes-and-their-products	1340	
38	en:cocoa-and-its-products	1323	
39	en:chips-and-fries	1317	

Next steps: [Generate code with top_tags](#) [New interactive sheet](#)

```
# Define keyword rules for each high-level bucket
# In this case, order matters. First match wins, so specific categories come before generic ones

def assign_primary_category(tags_string):
    if pd.isnull(tags_string):
        return 'Other'

    tags = tags_string.lower()

    category_rules = [
        (10, 'plant-based-foods-and-beverages', lambda x: 'plant-based-foods-and-beverages' in x),
        (1, 'plant-based-foods', lambda x: 'plant-based-foods' in x),
        (2, 'cereals-and-potatoes', lambda x: 'cereals-and-potatoes' in x),
        (3, 'snacks', lambda x: 'snacks' in x),
        (4, 'breads', lambda x: 'breads' in x),
        (5, 'dairies', lambda x: 'dairies' in x),
        (6, 'sweet-snacks', lambda x: 'sweet-snacks' in x),
        (7, 'fermented-foods', lambda x: 'fermented-foods' in x),
        (8, 'fermented-milk-products', lambda x: 'fermented-milk-products' in x),
        (9, 'desserts', lambda x: 'desserts' in x),
        (10, 'cereals-and-their-products', lambda x: 'cereals-and-their-products' in x),
        (11, 'dairy-desserts', lambda x: 'dairy-desserts' in x),
        (12, 'fermented-dairy-desserts', lambda x: 'fermented-dairy-desserts' in x),
        (13, 'yogurts', lambda x: 'yogurts' in x),
        (14, 'beverages', lambda x: 'beverages' in x),
        (15, 'condiments', lambda x: 'condiments' in x),
        (16, 'breakfasts', lambda x: 'breakfasts' in x),
        (17, 'fruits-and-vegetables-based-foods', lambda x: 'fruits-and-vegetables-based-foods' in x),
        (18, 'sauces', lambda x: 'sauces' in x),
        (19, 'meals', lambda x: 'meals' in x),
        (20, 'biscuits-and-cakes', lambda x: 'biscuits-and-cakes' in x),
        (21, 'breakfast-cereals', lambda x: 'breakfast-cereals' in x),
        (22, 'meats-and-their-products', lambda x: 'meats-and-their-products' in x),
        (23, 'confectioneries', lambda x: 'confectioneries' in x),
        (24, 'salty-snacks', lambda x: 'salty-snacks' in x),
        (25, 'frozen-foods', lambda x: 'frozen-foods' in x),
        (26, 'appetizers', lambda x: 'appetizers' in x),
        (27, 'spreads', lambda x: 'spreads' in x),
        (28, 'groceries', lambda x: 'groceries' in x),
        (29, 'cheeses', lambda x: 'cheeses' in x),
        (30, 'plant-based-beverages', lambda x: 'plant-based-beverages' in x),
        (31, 'biscuits', lambda x: 'biscuits' in x),
        (32, 'meats', lambda x: 'meats' in x),
        (33, 'vegetables-based-foods', lambda x: 'vegetables-based-foods' in x),
        (34, 'undefined', lambda x: 'undefined' in x),
        (35, 'beverages-and-beverages-preparations', lambda x: 'beverages-and-beverages-preparations' in x),
        (36, 'canned-foods', lambda x: 'canned-foods' in x),
        (37, 'legumes-and-their-products', lambda x: 'legumes-and-their-products' in x),
        (38, 'cocoa-and-its-products', lambda x: 'cocoa-and-its-products' in x),
        (39, 'chips-and-fries', lambda x: 'chips-and-fries' in x),
        (0, 'Other', lambda x: True)
    ]

    for rank, category, rule in category_rules:
        if rule(tags_string):
            return category

    return 'Other'
```

```

    ('Chocolate & Candy', ['chocolate', 'candies', 'candy', 'confectioner', 'cocoa']),
    ('Cookies & Biscuits', ['biscuit', 'cake', 'pastry', 'pastries']),
    ('Chips & Savory Snacks', ['chips-and-fries', 'crisps', 'salty-snacks', 'popcorn', 'pretzel']),
    ('Bread & Bakery', ['bread']),
    ('Dairy', ['dairy', 'cheese', 'yogurt', 'yoghurt', 'fermented-milk']),
    ('Beverages', ['beverage', 'drink', 'soda', 'juice']),
    ('Meat & Seafood', ['meat', 'seafood', 'fish', 'poultry']),
    ('Cereal & Bars', ['cereal', 'granola', 'muesli', 'breakfast']),
    ('Spreads & Condiments', ['spread', 'sauce', 'condiment', 'jam']),
    ('Sweet Snacks & Desserts', ['sweet-snacks', 'dessert', 'confection']),
    ('Plant-Based & Meals', ['plant-based', 'meal', 'ready-meal', 'legume']),
]

for category_name, keywords in category_rules:
    if any(kw in tags for kw in keywords):
        return category_name

return 'Other'

df_clean['primary_category'] = df_clean['categories_tags'].apply(assign_primary_category)

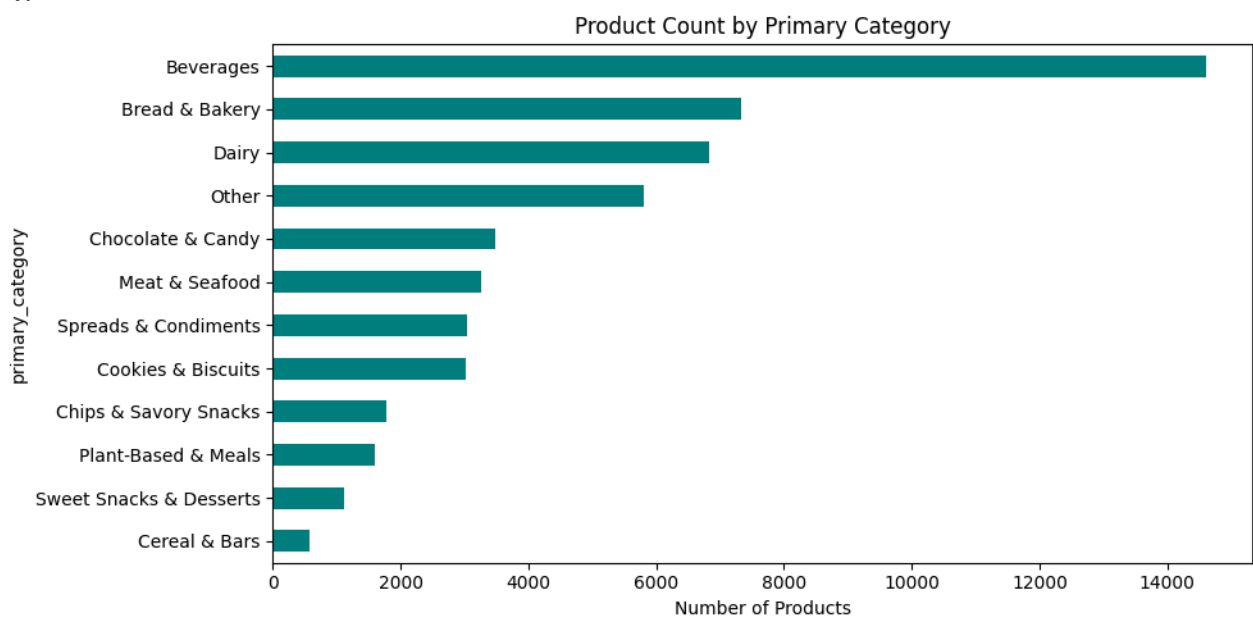
category_counts = df_clean['primary_category'].value_counts()
display(category_counts)

plt.figure(figsize=(10, 5))
category_counts.plot(kind='barh', color='teal')
plt.xlabel('Number of Products')
plt.title('Product Count by Primary Category')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

```

primary_category	count
Beverages	14599
Bread & Bakery	7323
Dairy	6822
Other	5811
Chocolate & Candy	3490
Meat & Seafood	3255
Spreads & Condiments	3043
Cookies & Biscuits	3015
Chips & Savory Snacks	1784
Plant-Based & Meals	1607
Sweet Snacks & Desserts	1116
Cereal & Bars	580

dtype: int64



Story 3: The "Nutrient Matrix" Visualization

```
# Scatter chart itself is will be built in Power BI, this is just to prepare the underlying data used to build the

# "Empty Quadrant" = high protein (that is greater than 10g/100g), low sugar ( less than 5g/100g)
empty_quadrant = df_clean[(df_clean['sugars_100g'] < 5) & (df_clean['proteins_100g'] > 10)].copy()

print(f"Empty Quadrant products: {len(empty_quadrant):,} of {len(df_clean):,} "
      f"({len(empty_quadrant)/len(df_clean)*100:.1f}%)")
display(empty_quadrant['primary_category'].value_counts())

# For each category: total products vs. how many ,lands in the Empty Quadrant
total_by_cat = df_clean.groupby('primary_category').size().rename('total_products')
quadrant_by_cat = empty_quadrant.groupby('primary_category').size().rename('in_quadrant')

gap_analysis = pd.concat([total_by_cat, quadrant_by_cat], axis=1).fillna(0)
gap_analysis['in_quadrant'] = gap_analysis['in_quadrant'].astype(int)
gap_analysis['pct_in_quadrant'] = (gap_analysis['in_quadrant'] / gap_analysis['total_products'] * 100).round(2)
gap_analysis = gap_analysis.sort_values('pct_in_quadrant')

display(gap_analysis)
```


Empty Quadrant products: 9,436 of 52,445 (18.0%)

	count
primary_category	
Meat & Seafood	2424
Beverages	2004
Dairy	1591
Other	1363
Bread & Bakery	1325
Plant-Based & Meals	269
Cookies & Biscuits	193
Chips & Savory Snacks	108
Spreads & Condiments	87
Chocolate & Candy	50
Cereal & Bars	11
Sweet Snacks & Desserts	11

dtype: int64

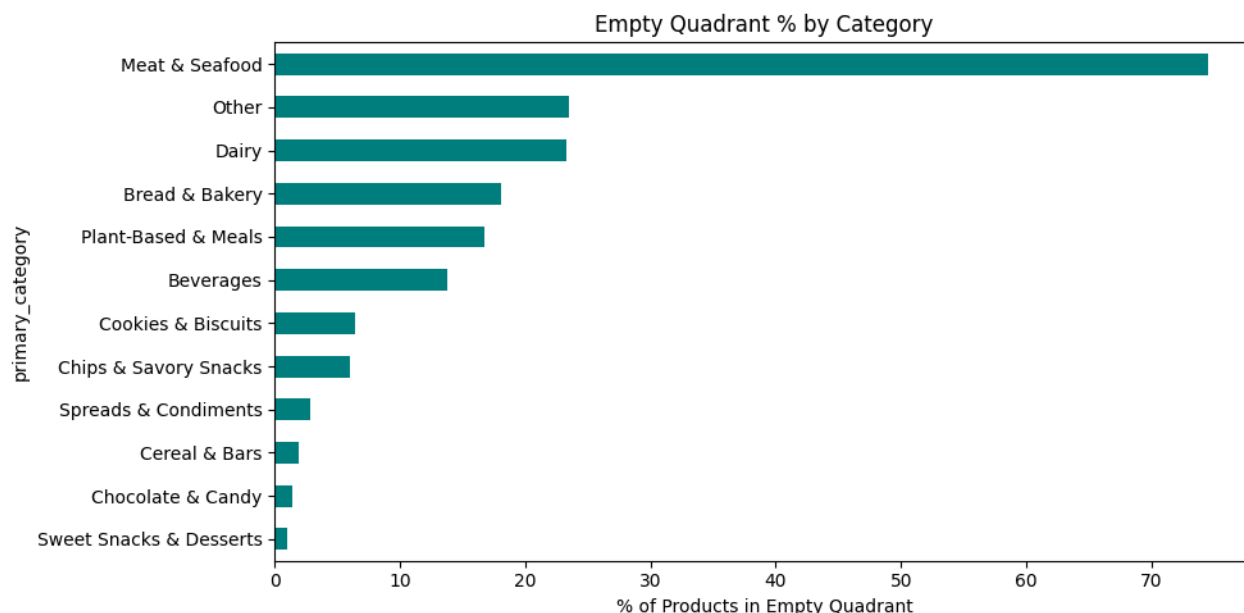
	total_products	in_quadrant	pct_in_quadrant
primary_category			
Sweet Snacks & Desserts	1116	11	0.99
Chocolate & Candy	3490	50	1.43
Cereal & Bars	580	11	1.90
Spreads & Condiments	3043	87	2.86
Chips & Savory Snacks	1784	108	6.05
Cookies & Biscuits	3015	193	6.40
Beverages	14599	2004	13.73
Plant-Based & Meals	1607	269	16.74
Bread & Bakery	7323	1325	18.09
Dairy	6822	1591	23.32
Other	5811	1363	23.46
Meat & Seafood	3255	2424	74.47

Next steps:

[Generate code with gap_analysis](#)

[New interactive sheet](#)

```
# Bar chart: Empty Quadrant % by category, sorted from lowest to highest
plt.figure(figsize=(10, 5))
gap_analysis['pct_in_quadrant'].plot(kind='barh', color='teal')
plt.xlabel('% of Products in Empty Quadrant')
plt.title('Empty Quadrant % by Category')
plt.tight_layout()
plt.show()
```



Story 4: The Recommendation

```
# Find the category with the biggest gap between total size and Empty Quadrant presence
top_opportunity = gap_analysis[gap_analysis['total_products'] > 500].sort_values('pct_in_quadrant').head(1)
display(top_opportunity)

# Suggested target numbers: median protein/sugar of the existing good products are used as a realistic benchmark
target_protein = empty_quadrant['proteins_100g'].median()
target_sugar = empty_quadrant['sugars_100g'].median()

print(f"Top opportunity category: {top_opportunity.index[0]}")
print(f"Suggested targets: {target_protein:.1f}g protein, less than {target_sugar:.1f}g sugar")
```

primary_category	total_products	in_quadrant	pct_in_quadrant
Sweet Snacks & Desserts	1116	11	0.99

Top opportunity category: Sweet Snacks & Desserts
Suggested targets: 16.1g protein, less than 0.9g sugar

Bonus User Story: The "Hidden Gem"

```
# Scoped to the recommended opportunity category, this will be the evidence that supports the recommendation.

target_category = top_opportunity.index[0]

good_products = empty_quadrant[
    (empty_quadrant['primary_category'] == target_category) &
    (empty_quadrant['ingredients_text'].notnull())
]

print(f"'{target_category}' products in the Empty Quadrant with ingredient text: {len(good_products)}")

protein_sources = ['whey', 'soy', 'pea protein', 'peanut', 'almond', 'egg', 'casein',
                   'milk protein', 'chicken', 'lentil', 'chickpea', 'oat', 'quinoa']

source_counts = {}
ingredients_lower = good_products['ingredients_text'].str.lower()

for source in protein_sources:
    count = ingredients_lower.str.contains(source, na=False).sum()
    source_counts[source] = count

source_df = pd.DataFrame(list(source_counts.items()), columns=['protein_source', 'count']) \
    .sort_values('count', ascending=False)

display(source_df.head(10))
print(f"\nTop 3 protein sources: {list(source_df.head(3)['protein_source'])}")
```